



中华人民共和国国家标准

GB/T 16262.4—2025/ISO/IEC 8824-4:2021

代替 GB/T 16262.4—2006

信息技术 抽象语法记法一(ASN.1) 第4部分:ASN.1 规范参数化

Information technology—Abstract Syntax Notation One(ASN.1)—
Part 4:Parameterization of ASN.1 specifications

(ISO/IEC 8824-4:2021, IDT)

2025-03-28 发布

2025-10-01 实施

国家市场监督管理总局 发布
国家标准化管理委员会

目 次

前言	III
引言	IV
1 范围	1
2 规范性引用文件	1
3 术语和定义	1
3.1 基本记法规范	1
3.2 信息客体规范	1
3.3 约束规范	1
3.4 附加定义	1
4 缩略语	2
5 约定	2
6 记法	2
6.1 赋值	2
6.2 参数化定义	2
6.3 符号表	3
7 ASN.1 词项	3
8 参数化赋值	3
9 引用参数化的定义	6
10 抽象语法参数	8
附录 A(资料性) 示例	10
A.1 使用参数化类型定义的示例	10
A.2 使用参数化定义以及信息客体类别的示例	10
A.3 有限的参数化类型定义的示例	11
A.4 参数化值定义的示例	12
A.5 参数化值集合定义的示例	12
A.6 参数化类别定义的示例	13
A.7 参数化客体集合定义的示例	13
A.8 参数化客体集合定义的示例	14
附录 B(资料性) 记法综述	15

前 言

本文件按照 GB/T 1.1—2020《标准化工作导则 第 1 部分：标准化文件的结构和起草规则》的规定起草。

本文件是 GB/T 16262《信息技术 抽象语法记法一(ASN.1)》的第 4 部分。GB/T 16262 已经发布了以下部分：

- 第 1 部分：基本记法规范；
- 第 2 部分：信息客体规范；
- 第 3 部分：约束规范；
- 第 4 部分：ASN.1 规范的参数化。

本文件代替 GB/T 16262.4—2006《信息技术 抽象语法记法一(ASN.1) 第 4 部分：ASN.1 规范的参数化》，与 GB/T 16262.4—2006 相比，除结构调整和编辑性改动外，主要技术变化如下：

- a) 更改了“ParameterList”中的“DummyReference”范围的表述(见 8.4, 2006 年版的 8.4)；
- b) 更改了“参数化赋值”和“参数化引用”的表述(见 8.7, 2006 年版的 8.7)。

本文件等同采用 ISO/IEC 8824-4:2021《信息技术 抽象语法记法一(ASN.1) 第 4 部分：ASN.1 规范的参数化》。

本文件做了下列最小限度的编辑性改动：

- 正文增加了对附录 A 和附录 B 的提及。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别专利的责任。

本文件由全国信息技术标准化技术委员会(SAC/TC 28)提出并归口。

本文件起草单位：中国电子技术标准化研究院、深圳赛西信息技术有限公司、中国科学院计算技术研究所、江苏赛西科技发展有限公司、联想(北京)有限公司、广东省工业边缘智能创新中心有限公司。

本文件主要起草人：王婷、苏静茹、郭雄、刘敏、杨宏、孙胜、张弛、蔡廷晓、马逸龙、卓兰、栾海晶、王博。

本文件及其所代替文件的历次版本发布情况为：

- 2006 年首次发布为 GB/T 16262.4—2006；
- 本次为第一次修订。

引 言

GB/T 16262《信息技术 抽象语法记法—(ASN.1)》旨在规范抽象语法记法,GB/T 16262 的编制基于 ISO/IEC 8824(所有部分)。GB/T 16262 拟由四个部分构成。

- 第 1 部分:基本记法规范。目的在于定义数据类型、值及数据类型的约束。
- 第 2 部分:信息客体规范。目的在于提供规定信息客体类别、信息客体和信息客体集合的记法。
- 第 3 部分:约束规范。目的在于提供规定用户定义的约束、表约束和内容约束的记法。
- 第 4 部分:ASN.1 规范参数化。目的在于定义 ASN.1 规范参数化记法。

信息技术 抽象语法记法一(ASN.1)

第4部分:ASN.1规范的参数化

1 范围

本文件定义了 ASN.1 规范的参数化记法。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中,注日期的引用文件,仅该日期对应的版本适用于本文件;不注日期的引用文件,其最新版本(包括所有的修改单)适用于本文件。

GB/T 16262.1—2025 信息技术 抽象语法记法一(ASN.1) 第1部分:基本记法规范 (ISO/IEC 8824-1:2021,IDT)

GB/T 16262.2—2025 信息技术 抽象语法记法一(ASN.1) 第2部分:信息客体规范 (ISO/IEC 8824-2:2021,IDT)

GB/T 16262.3—2025 信息技术 抽象语法记法一(ASN.1) 第3部分:约束规范 (ISO/IEC 8824-3:2021,IDT)

3 术语和定义

下列术语和定义适用于本文件。

3.1 基本记法规范

本文件使用 GB/T 16262.1—2025 中定义的术语。

3.2 信息客体规范

本文件使用 GB/T 16262.2—2025 中定义的术语。

3.3 约束规范

本文件使用 GB/T 16262.3—2025 中定义的术语。

3.4 附加定义

3.4.1

标准引用名 normal reference name

由“Assignment”方法而不是“ParameterizedAssignment”定义的、无参数的引用名。名称引用是一个完整的定义,并且在使用时无需向其提供实际的参数。

3.4.2

参数化引用名 parameterized reference name

使用参数化赋值定义的引用名,它引用不完整的定义,在使用时需向其提供实际的参数。

3.4.3

参数化类型 parameterized type

使用参数化类型赋值定义的类型,其组件是不完整的定义,在使用类型时需向其提供实际的参数。

3.4.4

参数化值 parameterized value

使用参数化赋值定义的值,其值没有完整地予以规定,在使用时需向其提供实际的参数。

3.4.5

参数化值集合 parameterized value set

使用参数化集合赋值定义的值,其值没有完整地予以规定,在使用时需向其提供实际的参数。

3.4.6

参数化客体类别 parameterized object class

使用参数化赋值定义的信息客体类别,其字段规范没有完整地予以规定,在使用时需向其提供实际的参数。

3.4.7

参数化客体 parameterized object

使用参数化赋值定义的信息客体,其组件没有完整地予以规定,在使用时需向其提供实际的参数。

3.4.8

参数化客体集合 parameterized object set

使用参数化赋值定义的信息客体集合,其客体没有完整地予以规定,在使用时需向其提供实际的参数。

3.4.9

可变约束 variable constraint

规定参数化抽象语法时采用的约束,取决于抽象语法的某个参数。

4 缩略语

下列缩略语适用于本文件。

ASN.1:抽象语法记法一(Abstract Syntax Notation One)

5 约定

本文件采用 GB/T 16262.1—2025 第 5 章定义的记法约定。

6 记法

本章概述了本文件定义的记法。记法综述见附录 B。

6.1 赋值

本文件定义了下列记法,此记法能用来替代“Assignment”(见 GB/T 16262.1—2025 第 13 章):
——ParameterizedAssignment(见 8.1)。

6.2 参数化定义

6.2.1 本文件定义了下列记法,此记法能用来替代“DefinedType”(见 GB/T 16262.1—2025 的 14.1):

——ParameterizedType(见 9.2)。

6.2.2 本文件定义了下列记法,此记法能用来替代“DefinedValue”(见 GB/T 16262.1—2025 中 14.1):

——ParameterizedValue(见 9.2)。

6.2.3 本文件定义了下列记法,此记法能用来替代“DefinedType”(见 GB/T 16262.1—2025 中 14.1):

——ParameterizedValueSetType(见 9.2)。

6.2.4 本文件定义了下列记法,此记法能用来替代“ObjectClass”(见 GB/T 16262.2—2025 中 9.2):

——ParameterizedObjectClass(见 9.2)。

6.2.5 本文件定义了下列记法,此记法能用来替代“Object”(见 GB/T 16262.2—2025 中 11.3):

——ParameterizedObject(见 9.2)。

6.2.6 本文件定义了下列记法,此记法能用来替代“ObjectSet”(见 GB/T 16262.2—2025 中 12.3):

——ParameterizedObjectSet(见 9.2)。

6.3 符号表

本文件定义了下列记法,此记法能用来替代“Symbol”(见 GB/T 16262.1—2025 中 13.1):

——ParameterizedReference(见 9.1)。

7 ASN.1 词项

本文件使用 GB/T 16262.1—2025 第 12 章规定的词项。

8 参数化赋值

8.1 与 GB/T 16262.1—2025 和 GB/T 16262.2—2025 规定的每一赋值语句有对应的参数化赋值语句。“ParameterizedAssignment”结构是:

```
ParameterizedAssignment ::=
    ParameterizedTypeAssignment          |
    ParameterizedValueAssignment         |
    ParameterizedValueSetTypeAssignment |
    ParameterizedObjectClassAssignment  |
    ParameterizedObjectAssignment       |
    ParameterizedObjectSetAssignment
```

8.2 除了在初始词项之后有一个“ParameterList”之外,每个“Parameterized<X> Assignment”与“<X> Assignment”有相同语法。因此初始项成为参数化引用名(见 3.4.2):

注 1: GB/T 16262.1—2025 要求在一个模块中所赋予的所有引用名,不管是否参数化,一定是明确的。

注 2: 如果值记法由参数化类型(或是参数的类型)控制,则参数化赋值内的值记法的有效性只能在参数化类型的实例化之后才能确定,并且可能对某些实例化有效,而对另一些实例化无效。

```
ParameterizedTypeAssignment ::=
    typereference
    ParameterList
    " : : ="
    Type
ParameterizedValueAssignment ::=
    valuereference
```

```

ParameterList
Type
"::="
Value
ParameterizedValueSetTypeAssignment::=
    typereference
    ParameterList
    Type
    "::="
    Value Set
ParameterizedObjectClassAssignment::=
    objectclassreference
    ParameterList
    "::="
    Object Class
ParameterizedObjectAssignment::=
    objectreference
    ParameterList
    DefinedObjectClass
    "::="
    Object
ParameterizedObjectSetAssignment::=
    objectsetreference
    ParameterList
    DefinedObjectClass
    "::="
    ObjectSet

```

8.3 “ParameterList”是括号之间的“Parameter”表

ParameterList::="{ "Parameter", "+" }"

每个“Parameter”由“DummyReference”和或许还有“ParamGovernor”组成：

Parameter::ParamGovernor": "DummyReference | DummyReference

ParamGovernor::Governor | DummyGovernor

Governor::Type | DefinedObjectClass

DummyGovernor::DummyReference

DummyReference::Reference

“Parameter”中的“DummyReference”可代表：

- 在没有“ParamGovernor”的情况，代表“Type”或“DefinedObjectClass”；
- 在应存在“ParamGovernor”的情况，代表“Value”或“ValueSet”；在“ParamGovernor”是“Governor”的情况，它应是“Type”；在“ParamGovernor”是“DummyGovernor”的情况，“ParamGovernor”的实际参数应是“Type”；
- 在应存在“ParamGovernor”的情况，代表“Object”或“ObjectSet”；在“ParamGovernor”是“Governor”的情况，它应是“DefinedObjectClass”；在“ParamGovernor”是“DummyGovernor”的情况，“ParamGovernor”的实际参数应是“DefinedObjectClass”。

“DummyGovernor”应是没有“Governor”的“DummyReference”。

8.4 出现在“ParameterList”中的“DummyReference”的范围是“ParameterList”本身,以及在“ParameterList”之后的“ParameterizedAssignment”部分。“DummyReference”在任何给定的实例化中,潜藏此范围中具有相同名称的其他“Reference”。

注:本条不适用于“NamedNumberList”“Enumeration”和“NamedBitList”中定义的“identifier”,因为它们不是“Reference”。“DummyReference”不隐藏“identifier”(见 GB/T 16262.1—2025 中 19.12 和 20.11)。

8.5 “DummyReference”在其范围内的用法应与其语法形式、何处应用、支配者一致,相同“DummyReference”的所有用法应彼此一致。

注:如果虚拟引用名称的语法形式有歧义(例如,在它是“objectclassreference”还是“typerreference”之间),歧义通常能在赋值语句的右边首次使用虚拟引用名称时解决。据此,虚拟引用名称的性质已知。然而虚拟引用的性质并不仅仅由赋值语句的右侧决定,当它反过来仅作为参数化引用中的实际参数使用时,在这种情况下,需通过检查这个参数化引用的定义来确定虚拟引用的性质。告知该符号的用户,这种做法会使 ASN.1 规范不那么清晰,建议提供足够的注释,为读者解释说明。

示例:

考虑有下列参数化客体类别赋值:

```
PARAMETERIZED-OBJECT-CLASS{ TypeParam, INTEGER: valueParam, INTEGER:
ValueSetParam} ::=
    CLASS{
        &.valueField1      TypeParam,
        &.valueField2      INTEGER DEFAULT valueParam,
        &.valueField3      INTEGER( ValueSetParam),
        &.ValueSetField    INTEGER DEFAULT{ ValueSetParam}
    }
```

为确定“ParameterizedAssignment”范围中的“DummyReference”的合适用法,且目的仅限于此,可考虑将“DummyReference”定义如下:

```
TypeParam ::= UnspecifiedType
valueParam INTEGER ::= unspecifiedIntegerValue
ValueSetParam INTEGER ::= { UnspecifiedIntegerValueSet}
```

其中:

- TypeParam 是代表“Type”的“DummyReference”。因此,凡是“typerreference”用作,例如固定类型值字段 valueField1 的“Type”时,可以使用 TypeParam。
- valueParam 是代表整数类型的值的“DummyReference”。因此,凡是能将整数值的“valuereference”用作,例如固定类型值字段 valueField2 的默认值时,可以使用 valueParam。
- ValueSetParam 是代表整数类型的值集合的“DummyReference”。因此,凡是能将整数值的“typerreference”用作,例如 valueField3 和 ValueSetField 的“ConstrainedSubtype”记法中的“Type”时,能使用 ValueSetParam。

8.6 在其范围内每个“DummyReference”应至少采用一次。

注:如果“DummyReference”没有出现,对应的“ActualParameter”对定义没有影响,并且将被简单的“丢弃”,而对用户来说,似乎有些规范正在发生。

包含对自身的直接或间接引用的“ParameterizedValueAssignment”“ParameterizedValueSetTypeAssignment”“ParameterizedObjectAssignment”“ParameterizedObjectSetAssignment”是不合法的。

8.7 给定一组参数化赋值(一个或多个)和通过一个或多个参数化赋值的一个参数化引用的递归路径,沿着该路径的所有参数化引用应满足以下条件:参数化引用中出现的每个实际参数,要么只包含一个虚设引用(在虚设引用之前或之后没有词项或注释),要么不包含任何虚设引用(见附录 A 的 A.3)。

8.8 在“ParameterizedType”“ParameterizedValueSet”或“ParameterizedObjectClass”的定义中,不应

产生对被定义项的循环引用,除非这种引用直接或间接标记成 OPTIONAL 或在通过对选择类型的引用产生的“ParameterizedType”和“ParameterizedValueSet”情况,至少其替代记法之一在定义中是非循环的。

8.9 “DummyReference”的支配者不应包括对另一个同样具有支配者的“DummyReference”的引用。

8.10 在参数化赋值中“::=”的右边不应仅由“DummyReference”组成。

8.11 “DummyReference”的支配者不应要求知道此“DummyReference”,或正被定义的参数化引用名。

8.12 当值或值集合作为实际参数提供给参数化类型时,要求此实际参数的类型与对应虚设参数的支配者兼容(详见 GB/T 16262.1—2025 中 C.6.2 和 C.6.3)。

8.13 在定义具有值或值集合虚设参数的参数化类型时,用来支配该虚设参数的类型应是这一类型,其所有值用于赋值右边使用虚设参数的各处都是有效的(详见 GB/T 16262.1—2025 中 C.6.5)。

9 引用参数化的定义

9.1 在“SymbolList”(在“Exports”或“Imports”)中,参数化定义应通过“ParameterizedReference”予以引用:

ParameterizedReference::=Reference | Reference"{"}"

其中,如 8.2 规定,“Reference”是“ParameterizedAssignment”中的第一个词项。

注:提供“ParameterizedReference”第一个替代记法只是帮助人们理解。两种替代记法具有相同意义。

9.2 除“Exports”或“Imports”以外,参数化定义应通过“Parameterized<X>”结构予以引用,此结构能用作对应“<X>”的替代记法:

ParameterizedType::=

SimpleDefinedType

ActualParameterList

SimpleDefinedType::=

ExternalTypeReference |

typereference

ParameterizedValue::=

SimpleDefinedValue

ActualParameterList

SimpleDefinedValue::=

ExternalValueReference |

valuereference

ParameterizedValueSetType::=

SimpleDefinedType

ActualParameterList

ParameterizedObjectClass::=

DefinedObjectClass

ActualParameterList

ParameterizedObjectSet::=

DefinedObjectSet

ActualParameterList

ParameterizedObject::=

DefinedObject

ActualParameterList

9.3 “Defined<X>”中的引用名应是“ParameterizedAssignment”中对其产生赋值的引用名。

9.4 使用“Defined<X>”替代记法的限制,在 GB/T 16262.1—2025 和 GB/T 16262.2—2025 中规定为正式引用名,同样适用于相应的参数化引用名。

注:大体上,限制如下:每个“Defined<X>”有两种替代记法,即“<X>reference”和“External<X>Reference”。前者在定义模块中使用,或如果已输入定义并且没有名称冲突时使用;后者在有所列的输入(未赞同),或如果被输入的名称与本地定义之间(也未赞同)或在输入之间有冲突时使用。

9.5 “ActualParameterList”是:

```
ActualParameterList ::=
    "{" ActualParameter "," "+" }"
Actual Parameter ::=
    Type |
    Value |
    ValueSet |
    DefinedObjectClass |
    Object |
    ObjectSet
```

9.6 对应“ParameterizedAssignment”中的每个“Parameter”应正好有个“ActualParameter”,并且以同样的顺序出现。对“ActualParameter”和支配者(如果有)的具体选择应通过此“Parameter”的语法形式和它出现在“ParameterizedAssignment”的环境的检查予以确定。“ActualParameter”的形式应是取代其范围内每处的“DummyReference”所要求的形式(见 8.4)。

示例:

例如,可以引用前面示例(见 8.5)的参数化客体类别定义如下:

```
MY-OBJECT-CLASS ::= PARAMETERIZED-OBJECT-CLASS{BIT STRING,123,{4 | 5 | 6}}
```

9.7 在确定正由使用参数化引用名的实例引用的实际类型、值、值集合、客体类别、客体或客体集合时,实际参数代替虚设引用名。

9.8 出现在“ActualParameter”中任一引用的含义和默认适用于也是如此出现的任何标记的标记,按照“ActualParameter”的标记环境而不是对应的“DummyReference”的标记环境予以确定。

注:因此,参数化,例如引用、选择类型和 COMPONENTS OF 的参数化,准确地说不是文字替换。

示例:

假设有下列模块:

```
M1 DEFINITIONS AUTOMATIC TAGS ::= BEGIN
    EXPORTS T1;
    T1 ::= SET{
        f1    INTEGER,
        f2    BOOLEAN
    }
END
M2 DEFINITIONS EXPLICIT TAGS ::= BEGIN
    IMPORTS T1 FROM M1;
    T3 ::= T2{T1}
    T2{X} ::= SEQUENCE{
        a    INTEGER,
        b    X
    }
END
```

END

9.8 意味着对 T3 的组件 f1(即@T3.b.f1)的标签进行隐式标记,因为虚拟参数 X 的标记环境即显式标记并不影响对实际参数 T1 的组件的标记。

假设有模块 M3:

```
M3 DEFINITIONS AUTOMATIC TAGS ::= BEGIN
    IMPORTS T1 FROM M1;
    T5 ::= T4{T1}
    T4{Y} ::= SEQUENCE{
        a    INTEGER,
        b    Y
    }
END
```

应用 GB/T 16262.1—2025 中 31.2.7 意味着 T5 的组件 b 的标记(即@T5.b)将显式地予以标记,因为虚设参数 Y 总是显式地予以标记,因此 T5 等效于:

```
T5 ::= SEQUENCE{
    a    [0]IMPLICIT INTEGER,
    b    [1]EXPLICIT SET{
        f1    [0]INTEGER,
        f2    [1]BOOLEAN
    }
}
```

而 T3 等效于:

```
T3 ::= SEQUENCE{
    a    INTEGER,
    b    SET{
        f1    [0]IMPLICIT INTEGER,
        f2    [1]IMPLICIT BOOLEAN
    }
}
```

10 抽象语法参数

10.1 GB/T 16262.2—2025 的附录 B 提供了 ABSTRACT-SYNTAX 信息客体类别,并建议用其定义抽象语法。一个示例是将抽象语法定义成一个 ASN.1 类型的值集合,而这 ASN.1 类型的外层未被参数化。相关示例见附录 A。

10.2 用来定义抽象语法的 ASN.1 类型予以参数化时,某些参数可作为定义抽象语法时的实际参数予以提供,而一些其他参数留作抽象语法本身的参数。

示例:

如果某一参数化类型已定义,叫做有两个虚设引用(假设第一个是某个已定义客体类别的客体集合,第二个是在一个边界内的整数值)的 YYY-PDU,那么:

```
yyy-Abstract-Syntax{INTEGER;bound} ABSTRACT-Syntax ::=
    {YYY-PDU{{ValidObjects},bound} IDENTIFIED BY{yyy5}}
```

定义其中已解决客体集合的参数化抽象语法,而 bound 留作抽象语法的一个参数。

抽象语法参数的使用方法是：

- a) 直接或间接用于约束的上下文中；
- b) 直接或间接用作最后用于约束上下文中的实际参数。

注：见 A.2 中的示例和 GB/T 16262.1—2025 中 H.5 中的示例。

10.3 其值集合视抽象语法一个或几个参数而定的约束是可变约束。在定义抽象语法之后确定这种约束(或许由国际标准化概要或在协议实现一致性声明中确定)。

注：如果在约束值规范包含的定义中的某处出现了抽象语法的参数,那么,此约束是可变约束。即使产生的约束的值集合与抽象语法的参数的实际值无关,它也是可变约束。

示例：

((1..3)EXCEPT a)UNION(1..3)的值总是 1..3,与 a 是什么值无关,若 a 是抽象语法的一个参数,它仍是可变约束。

10.4 形式上,可变约束不约束抽象语法中的值集合。

注：强力推荐希望留作抽象语法中可变约束的约束要有使用 GB/T 16262.1—2025 中 53.4 提供的记法的特殊规范。

附录 A

(资料性)

示例

A.1 使用参数化类型定义的示例

假设一名协议设计者常常需要携带具有一个或多个协议字段的鉴别符。这将按 BIT STRING 携带(位于此字段一侧)。没有参数化, Authenticator 需定义为 BIT STRING, 因此不管 authenticator 是否要出现, 需将它附加到文本以标识它应用于什么。换言之, 此设计者要遵守将具有鉴别符的任一字段能换成此字段和 authenticator 的 SEQUENCE 的约束。参数化机制为此工作提供便捷的方法。

首先, 我们定义参数化类型 SIGNED{}:

```
SIGNED{ ToBeSigned } ::= SEQUENCE
{
    authenticated-data      ToBeSigned,
    authenticator            BIT STRING
}
```

因此, 在协议的主体中, 此记法(作为示例)

```
SIGNED{ OrderInformation }
```

是下列的类型记法:

```
SEQUENCE
{
    authenticated-data      OrderInformation,
    authenticator            BIT STRING
}
```

进一步假设发送者作出对某些字段是否增加鉴别符的选择。这能通过产生可选的 BIT STRING 完成, 但是更好的方案(线上比特更少)是定义另一种参数化类型。

```
OPTIONALLY-SIGNED{ ToBeSigned } ::= CHOICE
{
    unsigned-data    [0]    ToBeSigned,
    signed-data      [1]    SIGNED{ ToBeSigned }
}
```

注: 如果编制者保证在使用参数化类型中没有一次产生 BIT STRING 的实际自变量(SIGNED 的类型), 则 CHOICE 中的标记不是必需的, 但是在防止规范其他部分中的差错是有用的。

A.2 使用参数化定义以及信息客体类别的示例

使用信息客体类别以收集抽象语法的所有参数。用此方法能使抽象语法的参数数目减少到收集类别的一个实例。能使用“InformationFromObject”生成式提取参数客体中的信息。

示例:

—这种类别的一个实例包含抽象语法 Message-PDU 的所有参数。

```
MESSAGE-PARAMETERS ::= CLASS(
    &.maximum-priority-level          INTEGER,
    &.maximum-message-buffer-size     INTEGER,
```



```

&.maximum-reference-buffer-size      INTEGER
}
WITH SYNTAX{
    THE MAXIMUM PRIORITY LEVEL IS&.maximum-priority-level
    THEMAXIMUM MESSAGE BUFFER SIZE IS&.maximum-message-buffer-size
    THE MAXIMUM REFERENCE BUFFER SIZE IS&.maximum-reference-buffer-size
}
--“ValueFromObject”生成式用来提取抽象语法参数“Param”中的值。这些值只能用于约束中。
--此外,此参数能一直被传送给另一个参数化类型。
Message-PDU{ MESSAGE-PARAMETERS:param } ::= SEQUENCE(
    priority-level INTEGER(0..param,&.maximum-priority-level),
    message BMPString(SIZE(0..param,&.maximum-message-buffer-size)),
    reference Reference{param}
)
Reference{ MESSAGE-PARAMETERS:param } ::=
    SEQUENCE OF IA5String(SIZE(0..param,&.maximum-reference-buffer-size))
--参数化抽象语法信息客体的定义。
--此抽象语法参数只能用于约束。
message-Abstract-Syntax{ MESSAGE-PARAMETERS:param }
    ABSTRACT-SYNTAX ::=
    {
        Message-PDU{param}
        IDENTIFIED BY {joint-iso-itu-t example(999) 0}
    }
类别 MESSAGE-PARAMETERS 和参数化抽象语法客体 message-Abstract-Syntax 使用如下:
--MESSAGE-PARAMETERS 的实例定义此抽象语法的参数值。
my-message-parameters MESSAGE-PARAMETERS ::= {
    THE MAXIMUM PRIORITY LEVEL IS 10
    THE MAXIMUM MESSAGE BUFFER SIZE IS 2000
    THE MAXIMUM REFERENCE BUFFER SIZE IS 100
}
--现在可用规定的所有可变约束定义此抽象语法。
my-message-Abstract-Syntax ABSTRACT-SYNTAX ::=
    message-Abstract-Syntax{ my-message-parameters}

```

A.3 有限的参数化类型定义的示例

当规定作为一个通用表形式的参数化类型时,规定此种类型会使产生的 ASN.1 记法是有限的。例如,我们规定:

```

List1{ElementTypeParam} ::= SEQUENCE{
    ElemElementTypeParam,
    next List1{ElementTypeParam} OPTIONAL
}

```

为了使用下列形式时是有限的:

```
IntegerList1 ::= List1{INTEGER}
```

产生的 ASN.1 记法正如它通常定义的解释:

```
IntegerList1 ::= SEQUENCE{
```

```

    elem INTEGER,
    next IntegerList1 OPTIONAL
  }

```

将此与下列定义比较：

```

List2{ElementTypeParam} ::= SEQUENCE{
    elemElementTypeParam,
    next List2{[0]ElementTypeParam} OPTIONAL
}

```

```

IntegerList2 ::= List2{INTEGER}

```

在此场合,产生的 ASN.1 记法是无限的：

```

IntegerList2 ::= SEQUENCE{
    elem INTEGER,
    next SEQUENCE{
        elem [0][0]INTEGER,
        next SEQUENCE{
            elem [0][0][0]INTEGER,
            next SEQUENCE{
                elem [0][0][0]INTEGER,
                next SEQUENCE{
                    --等等
                }OPTIONAL
            }OPTIONAL
        }OPTIONAL
    }OPTIONAL
}

```

A.4 参数化值定义的示例

如果定义参数化串值如下：

```

genericBirthdayGreeting{IA5String:name}IA5String := {"Happy birthday,",
name,"!!"}

```

那么以下两个字符串值是相同的：

```

greeting1 IA5String ::= genericBirthdayGreeting("John")
greeting2 IA5String ::= "Happy birthday,John!!"

```

A.5 参数化值集合定义的示例

如果定义两个参数化值集合如下：

```

QuestList1{IA5String:extraQuest}IA5String ::= {"Jack"|"John"|
extraQuest}
QuestList2{IA5String:ExtraQuest}IA5String ::= {"Jack"|"John"|
ExtraQuests}

```

那么下列值集合表示此相同的值集合：

```

SetofQuests1 IA5String ::= {QuestList1{"Jill"}}
SetofQuests2 IA5String ::= {QuestList2{{"Jill"}}}

```

SetofQuests3 IA5String::={"Jack"|"John"|"Jill"}

以及下列值集合表示此相同的值集合：

SetofQuests4 IA5String::={QuestList2{{"Jill"|"Mary"}}}

SetofQuests5 IA5String::={"Jack"|"John"|"Jill"|"Mary"}

注意，一个值集合总是在花括号中规定的，即使它是一个参数化值集合引用时也是如此。通过在对曾在值集合赋值中创建的“identifier”的引用中略去花括号或在对“ParameterizedValueSetType”的引用中略去花括号，此记法则是“Type”的记法，而不是值集合记法。

A.6 参数化类别定义的示例

下列参数化类别可用来定义包含不同类型的差错代码的差错类别。注意，ErrorCodeType 参数只能用作 ValidErrorCodes 参数的“DummyGovernor”。

```
GENERIC-ERROR{ErrorCodeType, ErrorCodeType: ValidErrorCodes}::=CLASS{
    &.errorCodeValidErrorCodes
}
WITH SYNTAX{
    CODE &.errorCode
}
```

能使用如下参数化类别定义来定义像同样的定义语法那样共享一些特性的不同类别：

```
ERROR-1::=GENERIC-ERROR{INTEGER, {1 | 2 | 3}}
ERROR-2::=GENERIC-ERROR{ErrorCodeString, {StringErrorCode}}
ERROR-3::=GENERIC-ERROR{EnumeratedErrorCode, {fatal | error}}
ErrorCodeString::=IA5String(SIZE(4))
StringErrorCodesErrorCodeString::={"E001" | "E002" | "E003"}
EnumeratedErrorCode::=ENUMERATED{fatal, error, warning}
```

因此定义类别按如下使用：

```
MY-Errors ERROR-2::={{CODE"E001"} | {CODE"E002"}}
fatalError ERROR-3::={CODE fatal}
```

A.7 参数化客体集合定义的示例

参数化客体集合定义 AllTypes 构成包含基本客体集合 BaseType 和以参数 AdditionalTypes 提供的附加客体集合的客体集合：

```
AllTypes{TYPE-IDENTIFIER: AdditionalTypes} TYPE-IDENTIFIER::=
{BaseTypes | AdditionalTypes}
BaseTypes TYPE-IDENTIFIER::={
    {BasicType-1 IDENTIFIED BY basic-type-obj-id-value-1} |
    {BasicType-2 IDENTIFIED BY basic-type-obj-id-value-2} |
    {BasicType-3 IDENTIFIED BY basic-type-obj-id-value-3}
}
```

参数化客体集合定义 ALLTypes 能使用如下：

```
MY-ALL-Types TYPE-IDENTIFIER::={AllTypes{
    {MY-Type-1 IDENTIFIED BY my-obj-id-value-1} |
    {MY-Type-2 IDENTIFIED BY my-obj-id-value-2} |
    {MY-Type-3 IDENTIFIED BY my-obj-id-value-3}
```

```
}}}
```

A.8 参数化客体集合定义的示例

在如下的参数化抽象语法定义中能使用 GB/T 16262.3—2025 中 A.4 中定义的类型：

--PossibleBodyTypes 是一种抽象语法的参数。

```
message-abstract-syntax { MHS-BODY-CLASS: PossibleBodyTypes } ABSTRACT-SYN-  
TAX ::= {
```

```
    INSTANCE OF MHS-BODY-CLASS({PossibleBodyTypes})
```

```
    IDENTIFIED BY {joint-iso-itu example(999) 1}
```

```
}
```

--此客体集合列出单一实例类型所有可能成对的值和 type-ids。

--此客体集合用作参数化抽象语法定义的实际参数。

```
MY-Body-Types MHS-BODY-CLASS ::= {
```

```
    {MY-First-Type IDENTIFIED BY my-first-obj-id} |
```

```
    {MY-Second-Type IDENTIFIED BY my-second-obj-id} |
```

```
}
```

```
my-message-abstract-syntax ABSTRACT-SYNTAX ::=
```

```
    message-abstract-syntax({My-Body-Types})
```

附录 B

(资料性)

记法综述

在 GB/T 16262.1—2025 中定义了下列词项并用于本文件：

```

typereference
valuereference
" ::= "
" { "
" } "
" , "

```

在 GB/T 16262.2—2025 中定义了下列词项并用于本文件：

```

objectclassreference
objectreference
objectsetreference

```

在 GB/T 16262.1—2025 中定义了下列生成式并用于本文件：

```

DefinedType
DefinedValue
Reference
Type
Value
ValueSet

```

在 GB/T 16262.2—2025 中定义了下列生成式并用于本文件：

```

DefinedObjectClass
DefinedObject
DefinedObjectSet
ObjectClass
Object
ObjectSet

```

本文件中定义了下列生成式：

```

ParameterizedAssignment ::=
    ParameterizedTypeAssignment      |
    ParameterizedValueAssignment      |
    ParameterizedValueSetTypeAssignment |
    ParameterizedObjectClassAssignment |
    ParameterizedObjectAssignment     |
    ParameterizedObjectSetAssignment
ParameterizedTypeAssignment ::=
    typereferenceParameterList " ::= " Type
ParameterizedValueAssignment ::=
    valuereferenceParameterList Type " ::= " Value
ParameterizedValueSetTypeAssignment ::=

```

```

    typereferenceParameterList Type"::="ValueSet
ParameterizedObjectClassAssignment::=
    objectclassreferenceParameterList"::="ObjectClass
ParameterizedObjectAssignment::=
    objectreferenceParameterListDefinedObjectClass"::="Object
ParameterizedObjectSetAssignment::=
    objectsetreferenceParameterListDefinedObjectClass"::="ObjectSet
ParameterList::="{ "Parameter", "+" }
Parameter::=ParamGovernor | DummyReference
ParamGovernor::=Governor | DummyGovernor
Governor::=Type | DefinedObjectClass
DummyGovernor::=DummyReference
DummyReference::=Reference
ParameterizedReference::=Reference | Reference "{ " " }"
SimpleDefinedType::=ExternalTypeReference | typereference
SimpleDefinedValue::=ExternalValueReference | valuereference
ParameterizedType::=SimpleDefinedTypeActualParameterList
ParameterizedValue::=SimpleDefinedValueActualParameterList
ParameterizedValueSetType::=SimpleDefinedTypeActualParameterList
ParameterizedObjectClass::=DefinedObjectClassActualParameterList
ParameterizedObjectSet::=DefinedObjectSetActualParameterList
ParameterizedObject::=DefinedObjectActualParameterList
ActualParameterList::="{ "ActualParameter", "+" }
ActualParameter::=Type | Value | ValueSet | DefinedObjectClass | Object | ObjectSet

```

中 华 人 民 共 和 国
国 家 标 准
信息技术 抽象语法记法一(ASN.1)
第 4 部分:ASN.1 规范参数化
GB/T 16262.4—2025/ISO/IEC 8824-4:2021

*

中国标准出版社出版发行
北京市朝阳区和平里西街甲 2 号(100029)

网址:www.spc.net.cn

服务热线:400-168-0010

2025 年 3 月第 1 版

*

书号:155066·1-78765

版权专有 侵权必究



GB/T 16262.4-2025